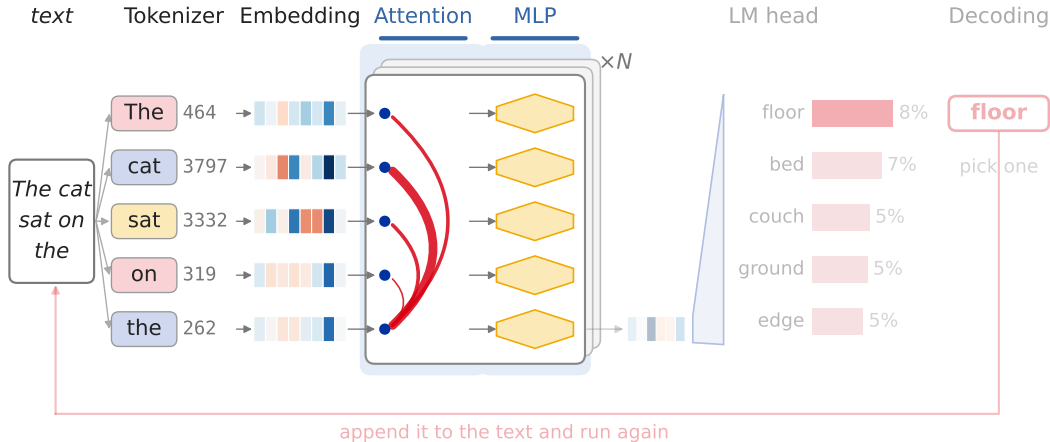


The Transformer Block

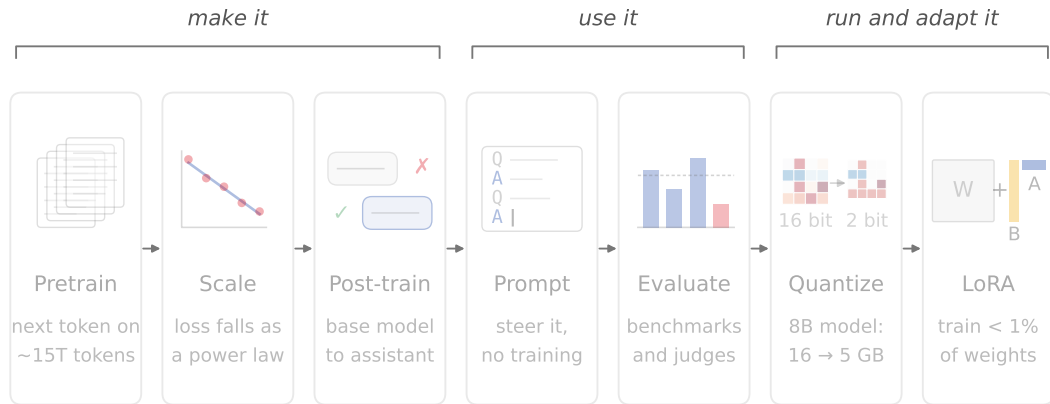
Many heads, position, and the residual stream

Where we are: one forward pass



Today: the rest of one layer - many heads, word order, and the MLP - and why the same layer can be stacked N times.

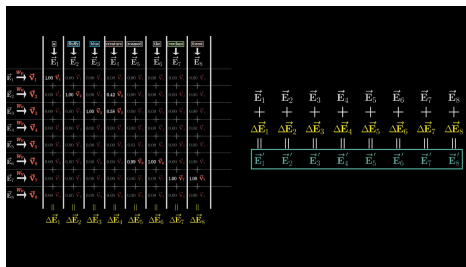
Where we are: the life of a model



Later: where these weights come from, and what you do with them once you have them.

Where we got to

Last lecture built **one head** of self-attention, end to end: every token emits a **query**, a **key** and a **value**; scores are $QK^\top / \sqrt{d_k}$; softmax turns them into weights; the output is a weighted sum of values.



3Blue1Brown

That head can already move a word's meaning toward its context. It is also, as we are about to see, **completely blind to word order**.

Three things stand between that and a Transformer

1. **One head is not enough.** A single pattern of who-looks-at-whom cannot capture syntax and meaning and position at once.
2. **Attention cannot see order.** We will prove this, not assert it.
3. **One layer is not a network.** Something has to hold the stack together and let gradients through.

Each of the three has a fix, and the three fixes bolted onto one head of attention *are* the Transformer block. That is the whole lecture.

Outline

Many heads

Position

The block

Many heads

One pattern of attention is never enough.

Run it h times, in parallel

Nothing about one head changes. You simply learn h independent sets of W^Q, W^K, W^V , run them at the same time, concatenate the outputs, and mix them with one more matrix W^O :

$$\begin{aligned}\text{MultiHead}(X) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O, \\ \text{head}_i &= \text{Attention}(XW_i^Q, XW_i^K, XW_i^V)\end{aligned}$$

The parameter count does not grow. Each head works in a smaller space: $d_k = d_{\text{model}}/h$. With $d_{\text{model}} = 512$ and $h = 8$, every head is 64-dimensional, so each W_i^Q is 512×64 . Eight of them is $8 \times 512 \times 64 = \mathbf{262,144}$ – exactly one 512×512 matrix. You are not adding capacity, you are **splitting** it into independent views.

And because the heads do not talk to each other until the concat, they are one more big batched matmul – the parallelism argument from last lecture survives intact.

Multi-headed attention

$W_Q^{(1)} W_Q^{(2)} W_Q^{(3)} W_Q^{(4)} W_Q^{(5)} W_Q^{(6)} W_Q^{(7)} W_Q^{(8)} W_Q^{(9)} \dots$

$W_K^{(1)} W_K^{(2)} W_K^{(3)} W_K^{(4)} W_K^{(5)} W_K^{(6)} W_K^{(7)} W_K^{(8)} W_K^{(9)} \dots$

$\downarrow W_V^{(1)} \downarrow W_V^{(2)} \downarrow W_V^{(3)} \downarrow W_V^{(4)} \downarrow W_V^{(5)} \downarrow W_V^{(6)} \downarrow W_V^{(7)} \downarrow W_V^{(8)} \downarrow W_V^{(9)} \dots$

$\uparrow W_V^{(1)} \uparrow W_V^{(2)} \uparrow W_V^{(3)} \uparrow W_V^{(4)} \uparrow W_V^{(5)} \uparrow W_V^{(6)} \uparrow W_V^{(7)} \uparrow W_V^{(8)} \uparrow W_V^{(9)} \dots$

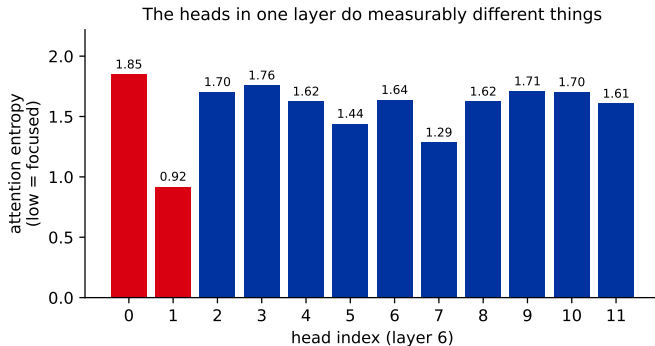
96



Multi-head attention: the same mechanism, many times over, each head free to learn a different pattern.

Do heads actually specialise, or is that a story?

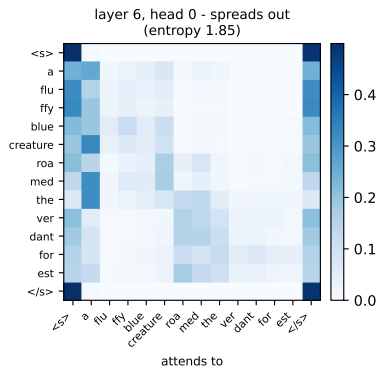
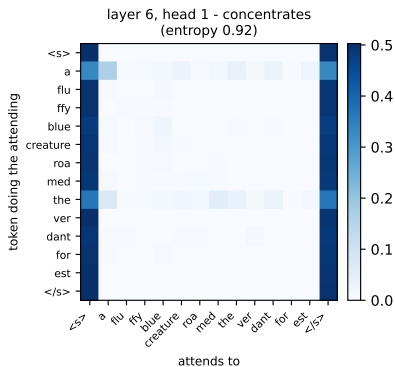
Easy to claim, so let us measure it. Take a real pretrained encoder, feed it one sentence, and pull out the actual attention weights from every head in one layer.



Attention entropy = $-\sum_j a_{ij} \log a_{ij}$, averaged over rows: low means a head concentrates on a few tokens, high means it hedges across many.

Two heads, same layer, same sentence

Two heads in the same layer, same sentence - real weights from intfloat/multilingual-e5-small



Head 1 concentrates (entropy **0.92**); head 0 spreads (entropy **1.85**) – in the *same* layer, on the *same* sentence, with the same input. Different views is not a metaphor; it is what the weights do.

What the specialisations look like

Across models and layers, heads reliably fall into recognisable jobs:

Kind of head	What it attends to
positional	the previous token, or the next one
syntactic	the verb belonging to this subject; the noun this adjective modifies
coreference	the name a pronoun refers to
lexical	the other half of a multi-token name (<i>Harry</i> $\leftrightarrow$ <i>Potter</i>)

Read these labels with some suspicion. Nobody assigned these jobs; they are patterns we recognise afterwards, and plenty of heads do nothing so tidy. The measured claim is the one on the last slide: the heads differ. The names are our interpretation.

Position

Attention has no idea which word came first.

Predict first

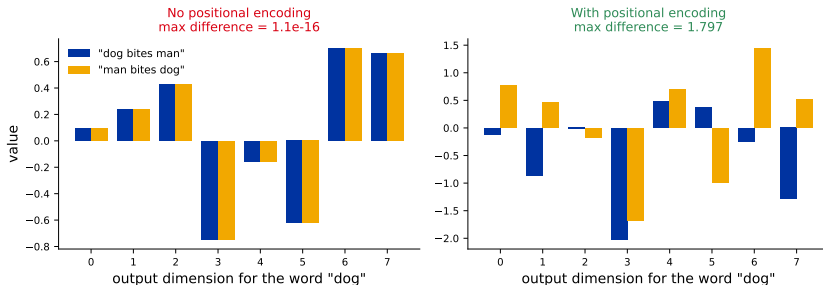
Take the same three word vectors in two orders:

“dog bites man” vs **“man bites dog”**

**Does one head of self-attention give
the word “dog” a different output vector?**

Commit to an answer.

No. Not slightly different – identical.



Left: the largest difference across the whole output vector is 1.1×10^{-16} – floating-point zero. Self-attention is **permutation-equivariant**: permute the inputs and the outputs come back in the same permuted order, unchanged. (Not *invariant* – the outputs do move; each one just stays attached to its own token.) A bag of words and a sentence are the same object to it.

Why that had to be true

Look back at the mechanism and there is nowhere for order to enter:

- ▶ $\mathbf{q}_i \cdot \mathbf{k}_j$ depends on the two *vectors*, not on i or j .
- ▶ The softmax is over a set of scores.
- ▶ The output is a *sum*, and addition does not care about order.

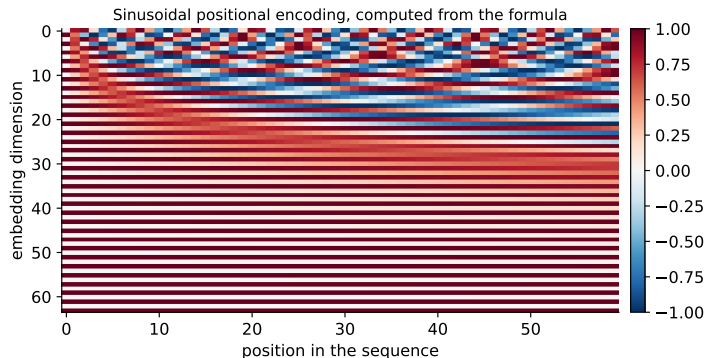
Concretely: in *both* sentences, “dog” attends to the same three vectors {dog, bites, man} and gets the same three weights. Only the order in which we *listed* them changed, and the sum does not notice.

An RNN got order for free – it processed tokens one at a time, so position was implicit in the computation. Attention threw the recurrence away to gain parallelism (L24), and **order was what it paid**. So order has to be put back by hand.

The fix is almost crude: add a position-dependent vector to each embedding before the first block, so that “dog in slot 1” and “dog in slot 3” are different inputs.

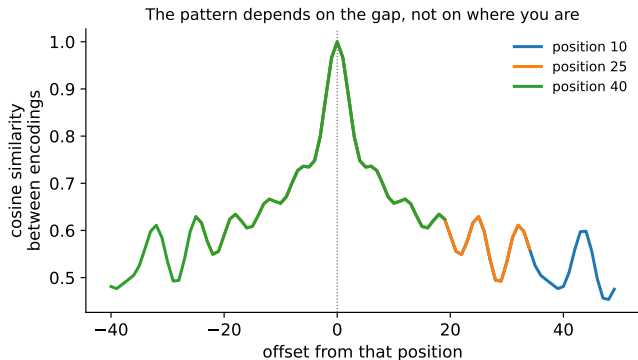
Sinusoidal positional encoding

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$



Each dimension is a sine wave; dimensions further down the embedding oscillate more slowly. Every position gets a distinct fingerprint, and no learned parameters are involved.

Why sinusoids rather than just the number *pos*



The similarity between two encodings depends on **how far apart** the positions are, not on where they sit. Measured here: the curves for positions 10, 25 and 40 agree to within **0.0000**. So a head can learn “attend three tokens back” once and reuse it everywhere – including at positions longer than anything it was trained on.

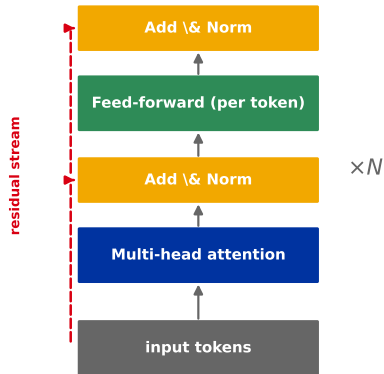
Modern models mostly use **RoPE** instead, which rotates **q** and **k** by an angle proportional to position. Same goal, better extrapolation.

The block

What you actually stack.

The Transformer block

One Transformer block



schematic

Multi-head attention – tokens exchange information. The *only* place in the whole block where tokens see each other.

Feed-forward network (FFN) – an ordinary MLP (**L14**) applied to each token separately, usually widening $4\times$ and back. Most of the parameters live here.

Add & Norm – the residual connection (**ResNet, L17**) plus LayerNorm: BatchNorm's idea from **L15**, but normalising across each token's *own features* instead of across the batch.

Three familiar pieces from three earlier chapters. The only genuinely new component in a Transformer is the attention – everything else you already met.

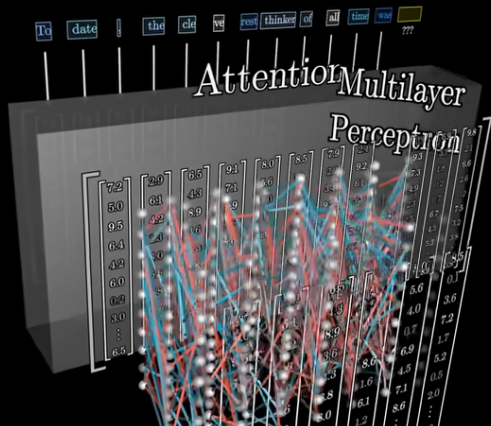
The residual stream is the real object

It is tempting to read the block as “attention, then an MLP”. A more useful reading: there is a **stream** of token vectors running straight through the network, and each sublayer *writes an update into it*.

$$\mathbf{x} \leftarrow \text{Norm}(\mathbf{x} + \text{Attention}(\mathbf{x})) \quad \text{then} \quad \mathbf{x} \leftarrow \text{Norm}(\mathbf{x} + \text{FFN}(\mathbf{x}))$$

Two consequences. **Gradients** reach the early layers unimpeded, which is why you can stack 96 of these – exactly the ResNet argument from L17. And every sublayer’s output is a **correction** to a running representation, not a fresh one, which is why the same token’s vector can accumulate meaning gradually with depth.

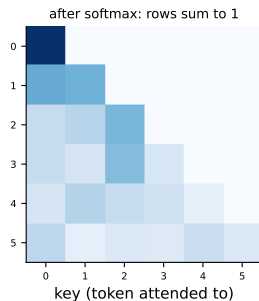
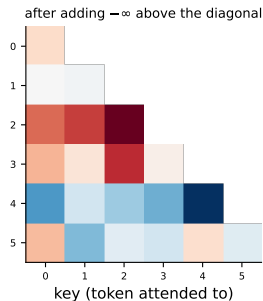
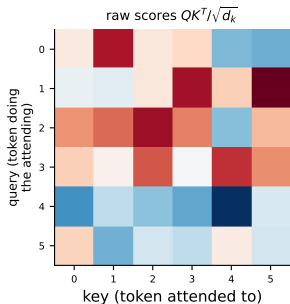
Read the order carefully: the norm comes *after* the addition, matching the diagram and the 2017 paper. Modern models move it **inside**, and we come back to why in three slides.



The flow through a block: attention lets tokens talk, then the MLP processes each one alone. Repeat.

Masking: how a decoder avoids reading the future

A model trained to predict the next token must not be allowed to look at it. The fix is one line: add $-\infty$ above the diagonal *before* the softmax.



$e^{-\infty} = 0$, so those weights become **exactly** zero – measured here as 0.000 attention paid to any future token – and each row still sums to one.

Why that trick matters more than it looks

Without masking, you would have to train on one prediction at a time: feed the first t tokens, predict token $t+1$, repeat.

With masking, a single forward pass over a sequence of length n produces n **training signals at once** – every prefix is being predicted simultaneously, and none of them can cheat.

This is a large part of why pretraining on internet-scale text is affordable at all. The causal mask is not a safety measure; it is a **throughput** decision that happens to enforce correctness.

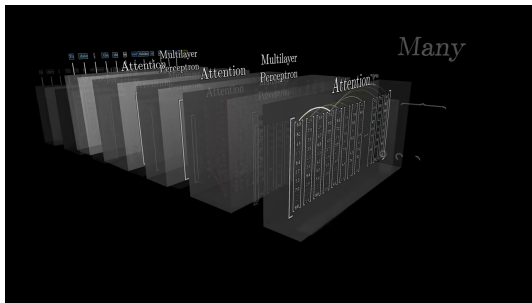
Modern block recipe

The 2017 block has been tuned. Three changes you will meet in every current model:

2017	Now	Why
Norm($\mathbf{x} + f(\mathbf{x})$)	$\mathbf{x} + f(\text{Norm}(\mathbf{x}))$	Pre-LN : trains stably, no warmup
LayerNorm	RMSNorm	cheaper, no mean subtraction
ReLU in the FFN	SwiGLU	gated, better quality per parameter

None of these changes the story. If you understand attention, residuals and a per-token MLP, you understand a 2026 model – the rest is engineering that moved a few percent and then stayed.

Now stack it



3Blue1Brown

Blocks are identical in shape and stacked N deep: 12 for the original BERT, 96 for GPT-3.

Each pass alternates **tokens talking** (attention) with **each token thinking alone** (the MLP).

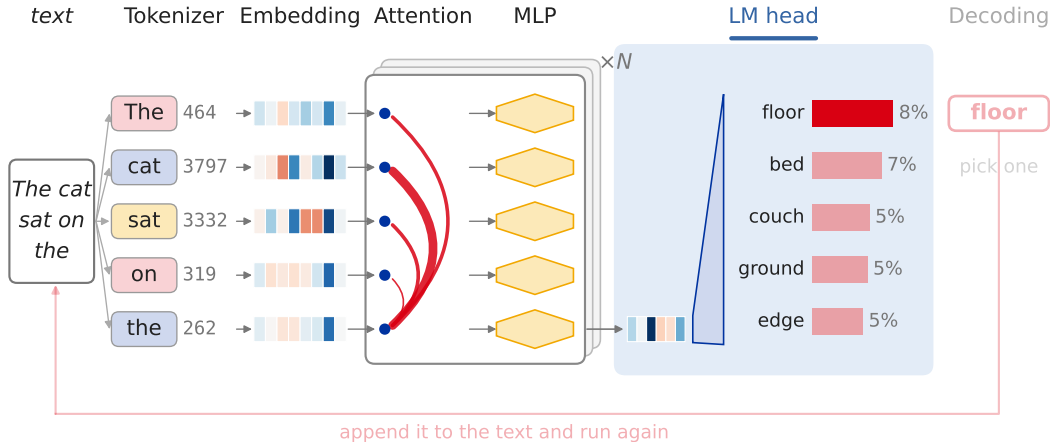
A token's vector starts as “the word *king*” and, layer by layer, accumulates everything the context implies – who this king is, what happened to him, what the sentence expects next.

Same block, many times. That is the entire architecture.

Recap

- ▶ **Multi-head** = the same attention run h times on *splits* of the embedding, then concatenated. No extra parameters, several independent views. Measured: 12 heads in one layer, attention entropy from 0.92 to 1.85.
- ▶ **Attention cannot see order** – proved, not asserted: reversing a sentence changed the output by 1.1×10^{-16} . Position must be added by hand.
- ▶ **Sinusoidal encodings** give every position a fingerprint whose similarity depends on distance, not absolute location. Modern models use RoPE for the same reason.
- ▶ **The block** = multi-head attention, then a per-token MLP, each wrapped in a residual and a norm. Attention is the only new part; residuals are L17 and norms are L15.
- ▶ **Causal masking** sets future weights to exactly zero, which turns one forward pass into n training examples.

Next on the map



Next (L26): what comes out at the far end - a distribution over the vocabulary - and the three ways the same block gets wired.

You can now build one.

What you cannot yet say is why *this* arrangement
beat everything else that was tried -
or what it is still bad at.

Next: the family, how it learns, and the honest limits.